

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

Helper Function for Text Cleaning:

Implement a Helper Function as per Text Preprocessing Notebook and Complete the following pipeline.

› Build a Text Cleaning Pipeline

[] ↪ 1 cell hidden

✓ Text Classification using Machine Learning Models

✓ Instructions: Trump Tweet Sentiment Classification

1. Load the Dataset

Load the dataset named "trump_tweet_sentiment_analysis.csv" using `pandas`. Ensure the dataset contains at least two columns: "text" and "label".

2. Text Cleaning and Tokenization

Apply a text preprocessing pipeline to the "text" column. This should include:

- Lowercasing the text
- Removing URLs, mentions, punctuation, and special characters
- Removing stopwords
- Tokenization (optional: stemming or lemmatization)
- "Complete the above function"

3. Train-Test Split

Split the cleaned and tokenized dataset into **training** and **testing** sets using `train_test_split` from `sklearn.model_selection`.

4. TF-IDF Vectorization

Import and use the `TfidfVectorizer` from `sklearn.feature_extraction.text` to transform the training and testing texts into numerical feature vectors.

5. Model Training and Evaluation

Import **Logistic Regression** (or any machine learning model of your choice) from `sklearn.linear_model`. Train it on the TF-IDF-embedded training data, then evaluate it using the test set.

- Print the **classification report** using `classification_report` from `sklearn.metrics`.

#Necessary Imports:

```
import pandas as pd
import re
import nltk
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

#Necessary downloads:

```
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

 [nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True
```

#1. Load the dataset

```
data = pd.read_csv("/content/drive/MyDrive/Artificial Intelligence/Week8/trum_tweet_sentiment_analysis.csv", encoding="ISO-8859-1")
data.head()
```

	text	Sentiment	
0	RT @JohnLeguizamo: #trump not draining swamp b...	0	
1	ICYMI: Hackers Rig FM Radio Stations To Play A...	0	
2	Trump protests: LGBTQ rally in New York https:...	1	
3	"Hi I'm Piers Morgan. David Beckham is awful b...	0	
4	RT @GlennFranco68: Tech Firm Suing BuzzFeed fo...	0	

Step 2: Text Cleaning and Tokenization

```
def text_cleaning_pipeline(dataset, rule="lemmatize"):
    data = dataset.lower()
    data = re.sub(r"http\S+|www\S+|https\S+", '', data) # Remove URLs
    data = re.sub(r'@\w+', '', data) # Remove mentions
    data = re.compile("[\"
        u\"\\U0001F600-\\U0001F64F\"
        u\"\\U0001F300-\\U0001F5FF\"
        u\"\\U0001F680-\\U0001F6FF\"
        u\"\\U0001F1E0-\\U0001F1FF\"
        u\"\\U00002702-\\U000027B0\"
        u\"\\U000024C2-\\U0001F251\"
        \"]+", flags=re.UNICODE).sub(r' ', data) # Remove emojis

    data = re.sub(r'[^\a-zA-Z0-9\s]', '', data) # Remove punctuation/special chars
    tokens = data.split()
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    if rule == "lemmatize":
        lemmatizer = WordNetLemmatizer()
        tokens = [lemmatizer.lemmatize(word) for word in tokens]
    elif rule == "stem":
        from nltk.stem import PorterStemmer
        stemmer = PorterStemmer()
        tokens = [stemmer.stem(word) for word in tokens]
    else:
        print("Pick between lemmatize or stem")

    return " ".join(tokens)
```

Apply cleaning to the text column

```
data["clean_text"] = data["text"].apply(lambda x: text_cleaning_pipeline(x, rule="lemmatize"))
```

3. Train-Test Split

```
X = data["clean_text"]
y = data["Sentiment"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4. TF-IDF Vectorization

```
vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

5. Model Training and Evaluation

```
model = LogisticRegression()
model.fit(X_train_vec, y_train)
y_pred = model.predict(X_test_vec)
```

```
⚠ /usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:

```
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression  
n_iter_i = _check_optimize_result(
```

```
# Print classification report  
print(classification_report(y_test, y_pred))
```